gtfs-ingest: What Changed

Within-feed station deduplication and stops-only CSV sources (MiBici) — code path, measured impact and open questions

Víctor Rodríguez

October 25, 2026

Uncommitted working tree on top of commit `1df490a` · line numbers refer to that tree · figures from the run `out_mibici_v2` (2 GTFS feeds + MiBici, 300 m proximity)

The change set at a glance

Three features, plus docs build and docstring fixes

33
files modified

+1,410 / −131
lines (tracked files)

+26
new tests (dedupe, CSV)

1 STOPS-ONLY CSV

A plain `id, name, latitude, longitude` file (MiBici: 484 docks) becomes a *feed* that contributes stop geometry only.

- new `stops_csv.py` (324 lines)
- `FeedSource.kind = "csv"`
- status `stops_only` in every phase

2 WITHIN-FEED DEDUPE

One feed publishing a station twice (`SITEUR_002_018` vs `ST_02_018`) is now merged — on shared routes only.

- new CLI flag, default 50 m
- new column `match_kind`
- DB migration via `ALTER TABLE`

3 UNSERVED NEIGHBOURS

A nearby stop with no serving line is now *listed* in `near_stations` with line `-`, instead of silently dropped.

- `UNSERVED_LINE = "-"`
- LEFT join in `_near_entries`
- makes docks visible to bus stops

Also: Sphinx docs CI job with `-W`, a `docs` extra in `pyproject.toml`, RST-safe docstrings across 10 modules, `source_kind` column in the Phase 1 report.

Where the diff lands

Every touched module, grouped by the feature it serves

Stops-only CSV

- `stops_csv.py` **new**
- `loader.py` +87
- `models.py` +55
- `validators.py` +31
- `pipeline_phase1.py`, `report.py`
- `cli.py` (help text)

Dedupe

- `analysis/dedupe.py` +117
- `analysis/storage.py` +9
- `cli.py` +33
- `conflation/pipeline_phase3.py`

Both / shared

- `analysis/pipeline_phase2.py` +107
- `analysis/near_stations.py` +46

Docs & hygiene

- `.github/workflows/tests.yml`
- `pyproject.toml`, `.gitignore`
- README, WORKFLOW, `PIPELINE_GUIDE`
- docstrings: headways, osm_match, ...
- 11 test files updated

Stops-only CSV

How 484 MiBici docks enter the pipeline

Discovery · loading · the `is_stops_only` branch
which outputs the docks reach — and which they cannot

One predicate drives every branch

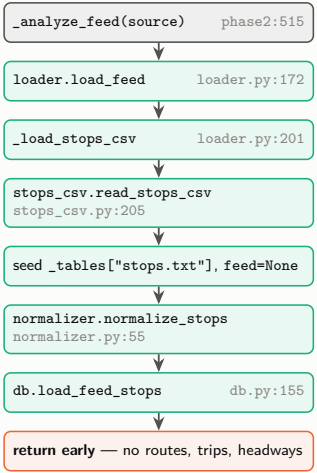
`FeedSource.is_stops_only` `models.py:155-163`

- **Discovery** tags an explicitly named `.csv` as `kind="csv"` `loader.py:94-95`
- A **directory scan never picks up CSVs** `loader.py:98-106` — otherwise a previous run's `near_stations.csv` would re-enter as a second network
- `feed_id` from the file stem: `mibici.csv` → `mibici` `loader.py:55`
- `is_stops_only` is simply `kind == "csv"`; LoadedFeed mirrors it `models.py:211`

Phase	What the branch does
1 ingest	INFO `stops_only_source` replaces 5 missing-file ERRORs `validators.py:135`
2 analyze	stops only, early return, status `stops_only` `pipeline_phase2.py:315`
2 analyze	excluded from `--common-date` `pipeline_phase2.py:486`
3 interch.	stops cluster, no routes `pipeline_phase3.py:159`

From `mibici.csv` to normalized stops

Per-feed path inside `_analyze_feed`



Inside `read_stops_csv`

- `_decode`: `utf-8-sig` first (strips the Excel BOM), then cp1252 :131
- each delimiter candidate :241: parse as strings :157, `resolve_header` via alias table :99; first full match wins
- rename + keep canonical columns :285 — `dpcapacity` **is dropped here**
- decimal-comma WARNING :289

Inside `normalize_stops`

- coordinates to float, invalid rows dropped :122–132
- duplicate ids dropped :135
- `stop_uid = "mibici:" + id` :143

`LoadedFeed.table()` checks the seeded cache before partridge, so validators and the normalizer need no special case.

The branch itself

`pipeline_phase2.py:315–334` — placed *before* route filter and service-date logic

```
315  if source.is_stops_only:
316      data.stops, _ = normalizer.normalize_stops(raw)
317      result.status = "stops_only"
         # service_date stays None -> not part of
         # the cross-feed date-coherence check
320      result.notes.append("stops-only CSV source: ...")
325      if config.mode_codes or config.route_specs:
329          result.notes.append(
                 "--transport-mode/--route not applicable")
332      if engine is not None:
333          db.load_feed_stops(engine, feed_id, data.stops)
334      return result, data
```

Why so early?

Every service-date path below would raise inside partridge (no calendar). And `data.stops` is normally set near the *end* of the function — a feed that leaves early would vanish from `all_stops`, PostGIS, `near_stations`, the GeoPackage and the station network.

Filters never delete stops

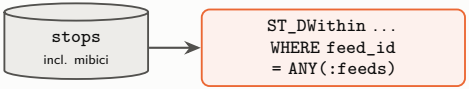
`--transport-mode`/`--route` only add a note: a dock has no route attribute to match.

A bug the dry runs could not show

Which feeds feed the spatial queries `pipeline_phase2.py:552-558`

BEFORE

```
analyzed_feed_ids = [status == "analyzed"]
```



mibici rows sit in PostGIS but match no pair ⇒ docks get **no neighbours** in full runs. Dry runs use `proximity_pairs_local(all_stops)` and ignore the list.

AFTER

```
geographic_feed_ids = [status in ("analyzed",
"stops_only")]
```

Used by all three DB-side spatial steps:

- `proximity_pairs_postgis` :556
- `walk_network.network_distances` :585
- `storage.fetch_osm_matches` :609

The schedule-based date-coherence check still uses `status == "analyzed"` only :523–527.

Where the docks show up — and where they cannot

Root cause of everything: docks are in `all_stops`, never in `route_stops` / `routes`

Stage	Docks?	Why	Code
`all_stops`, PostGIS `stops`	yes	stops-only branch	`phase2:315-334`
proximity pairs	yes	`geographic_feed_ids`	`phase2:552-558`
route connectivity edges	no	every join is on `route_stops`	`connectivity.py:248`
headways, same-line stations	no	no trips / stop_times	—
`near_stations.csv` rows	yes	one row per stop; `stop_line_ids` empty	`near_stations.py:181`
. . . as someone's neighbour	yes	line rendered as –	`near_stations.py:308`
station network	nodes + walk	no `in_vehicle` arcs	`station_network.py:99,139`
GeoPackage	yes	stops layer + near_links	`geopackage.py:116,139`
Phase 3 clusters	yes	empty mode set, never multimodal	`phase3:159`

484
docks in every stop output

5,756
station-network edges touching a dock

0
route edges from a dock

Unserved neighbours are now listed, not dropped

`_near_entries` near_stations.py:272–321

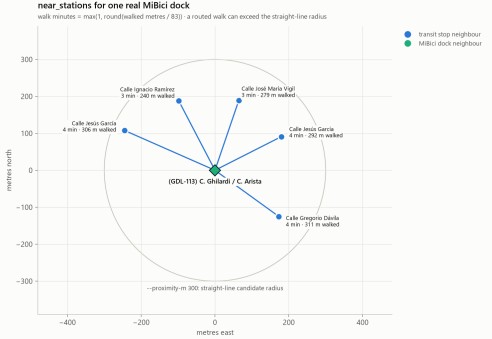
BEFORE inner join: a neighbour without a line vanishes

```
entries = both.merge(lines_at, on="near_uid")
# bus stop near a dock:
[(1, Stop, 10, 1)]          # dock missing
```

AFTER LEFT join + sentinel :303–309

```
entries = both.merge(served, on="near_uid",
                     how="left")
entries["line"] = entries["line"].fillna("-")
# bus stop near a dock:
[(13866, (GDL-001) C. Epigmenio..., -, 3),
 (8782, Avenida 16 de Septiembre, C05, 3)]
```

- early return now needs only `stop_pairs` :287 — a CSV-only run still gets lists
- the dock's *own* `system` / `stop_line_ids` stay empty
- quality sidecar: such a neighbour counts as `straight_line`, not `none`



8,660

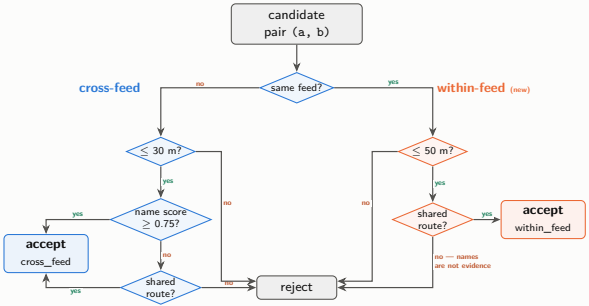
of 14,349 rows list ≥1 neighbour with line –

Deduplication

Cross-feed (always) + within-feed (new)

The accept rules · `find_equivalences` line by line · grouping how the groups propagate to every output

Two rules, deliberately different evidence



Motivating case — one feed, two id schemes, same place:

`SITEUR_002_018` ↔ `ST_02_018` **0.0 m**

- rendered the light-rail line as two disjoint chains
- names cannot merge a within-feed pair — one feed reuses names for corridors and plazas
- `shared_routes` counts normalized `route_short_names`, so it is **always 0 for a stops-only source**
- each accepted pair records `match_kind`

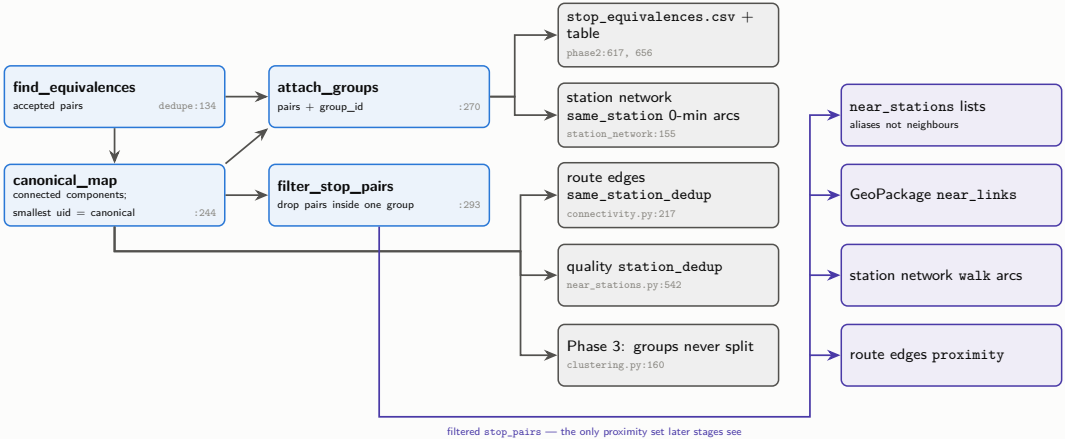
`find_equivalences`, line by line

`analysis/dedupe.py:134-241` — called from `pipeline_phase2.py:566` and `pipeline_phase3.py:236`

Lines	Step	Effect on MiBici docks
173–174	one spatial pass at `max(30, 50)` m with `proximity_pairs_local` (always local UTM, independent of the run's proximity pairs)	candidates like any other stop
178–185	join `feed_id`, `stop_name` onto both ends	names like `(GDL-001) C. ...`
186–191	**scope gate**: cross-feed $\leq$ 30 m; same-feed $\leq$ 50 m only if enabled	dock↔dock $\leq$ 50 m, dock↔bus $\leq$ 30 m
196	`match_kind` = `within_feed` / `cross_feed`	—
197–199	`name_similarity`: NFKD, casefold, token sort, `SequenceMatcher` :90,105	scored normally
200–206	`_route_name_sets` :117: route short names per stop; `shared_routes` = $\lvert A \cap B \rvert$	**always 0** (no entry)
210–214	**accept**: within = route only; cross = name $\geq$ 0.75 **or** route	dock↔dock never; dock↔bus **only via name**
222–229	audit `score` against the radius that admitted the pair	—
230–240	sort by pair key, log counts per kind	—

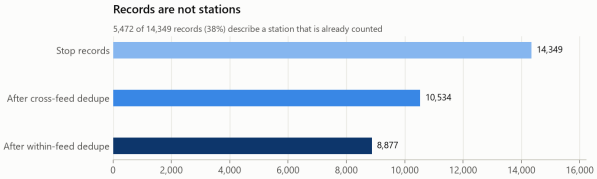
From accepted pairs to every output

`pipeline_phase2.py:563–573` and the consumers downstream



Measured: records versus stations

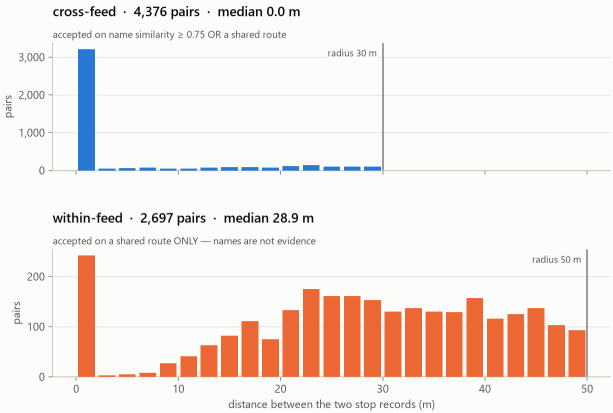
Same feeds, same run — only the dedupe rules differ



	`out_mibici`	`_v2`
rules	cross	+ within
accepted pairs	4,376	7,073
groups	1,406	2,572
stops merged	5,221	8,044
stations	10,534	**8,877**
`same_station_dedup` edges	1,587	1,762
MiBici stops merged	0	0

stations = records − merged + groups
= 14,349 − 8,044 + 2,572

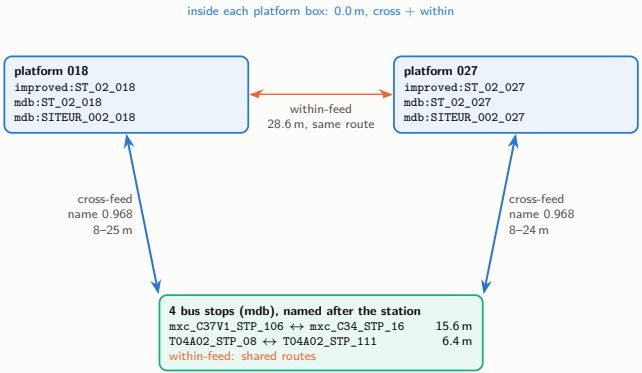
The two kinds look nothing alike



- **Cross-feed: 4,376 pairs, median 0.0 m** — the two feeds share coordinates.
- **Within-feed: 2,697 pairs, median 28.9 m** — spread over the whole 50 m radius.
- A true *duplicate record* (`SITEUR_*` vs `ST_*`) sits at 0.0 m. Most of the 20–50 m mass is something else → next slides.

Chaining: how a 10-stop group forms

Largest group in `out_mibici_v2` (group 1126, 25 pairs) — SiTren Line 2, stations 018 / 027

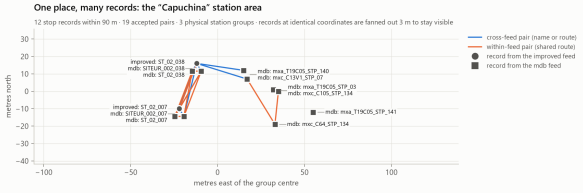


- `canonical_map` takes **connected components**
  `dedupe.py:259–261`
- A∼B (cross) and B∼C (within) ⇒ A, B, C are one station, though A and C were never compared
- Result: both direction platforms **plus** four bus stops named after the station

Is that one "station"?

For counting places: arguably yes. For transfers and direction-level analysis: it is an interchange, not an alias.

Opposite-direction platforms merge too



Typical within-feed pairs in the audit file:

pair	m
MM_A03_10 ↔ MM_A03_25	28.9
MM_A03_11 ↔ MM_A03_24	26.9
MM_A03_13 ↔ MM_A03_22	34.2
ST_02_018 ↔ ST_02_027	28.6
MM_001 ↔ MM_054	6.0

Stop k pairs with stop $N{+}1{-}k$: the outbound and return stops of one route, across the street.

The rationale has a gap

"Consecutive stops sit hundreds of metres apart" is true — but the same route's stop *on the other side of the street* is 20–40 m away and shares every route.

Open issues

Found while reviewing the diff and the outputs

Defaults and reporting · MiBici edge cases · routing side effects

none are crashes — all change numbers the paper reports

Findings

	Issue	Detail	Where
HIGH	Within-feed merges direction pairs	most 20–50 m pairs are opposite platforms; the 8,877 "stations" depend on that definition	`dedupe.py:210`
HIGH	CLI default ≠ config default	CLI turns within-feed **on** (50); `AnalysisConfig` says `None` = off, "historical behaviour"	`cli.py:127`, `phase2:113`
MED	Report omits the within-feed radius	header prints "on (30 m)" only; JSON has no field for it	`phase2:676, 735`
MED	Docks *can* merge with bus stops	cross-feed name rule applies to docks; 0 merges is luck ("(GDL-001)" prefix), not design. Docs say "never merged"	`dedupe.py:212`
MED	Free U-turns in routing	every equivalence, within-feed included, becomes a 0-min `same_station` arc	`station_network:155`
LOW	Docks extend walking paths	stop → dock → stop chains two 300 m walks between stops not within 300 m	`station_network:139`
LOW	`ID` depends on `--feeds` order	`ID = 1..N` over `all_stops`; listing the CSV first shifts every GTFS ID	`near_stations.py:187`
LOW	Unserved GTFS stops now listed	parent stations and stops with no service on the analyzed day also render as –, e.g. `(2250, Avenida 16 de Septiembre, –, 2)`	`near_stations.py:308`

Decisions to make

DEFINITION

Does a "station" in the paper include **both directions** of a route?

- **Yes** → keep 50 m; state it in the methods; report 8,877.
- **No** → a radius of ∼5 m keeps the 0 m duplicate records and drops the direction pairs; or accept only pairs sharing a `direction_id`.

MIBICI

- Exclude stops-only sources from cross-feed matching explicitly (a dock is never an alias of a bus stop) and add a test for it.
- Decide whether docks belong in the station network as walk waypoints.
- Keep `dpcapacity` if bike-share supply ever enters the analysis.

CONSISTENCY

- Align the CLI and `AnalysisConfig` defaults (and the comment).
- Add `dedupe_within_feed_distance_m` to `AnalysisReport` and the Markdown header.
- Fix the "never merged into anything" sentence in README and the dedupe docstring.
- Rename `test_same_feed_pair_is_never_an_alias` — true only for the function default.

Already solid

CSV header resolution and error messages, the early stops-only branch, `geographic_feed_ids`, `match_kind` audit column with DB migration, Sphinx `-W` in CI.

Tests and CI added with the change

Dedupe `tests/test_dedupe.py`, +8

- within-feed is off by default (function level)
- same-route pair merged; same-name-only pair rejected
- own radius respected; score stays in range past 30 m
- cross-feed still matches on name with within-feed on
- stops-only source never merged *within* feed
- `match_kind` survives `attach_groups`

Not covered yet

- opposite-direction platforms of one route
- a dock matching a bus stop by name (cross-feed)

Stops CSV `tests/test_stops_csv.py`, 18 tests

- aliases, accents, case; unknown and duplicate columns
- BOM, UTF-16, cp1252; `,` `;` `\t` `|` delimiters
- unresolved / partial / headerless headers produce findings
- decimal commas warned; values stay strings

Pipeline and others

- loader, validators, normalizer, pipeline, connectivity, near_stations, DB, Neo4j updated

CI `.github/workflows/tests.yml`

- new docs job: `sphinx -W --keep-going`, intersphinx off, HTML artifact